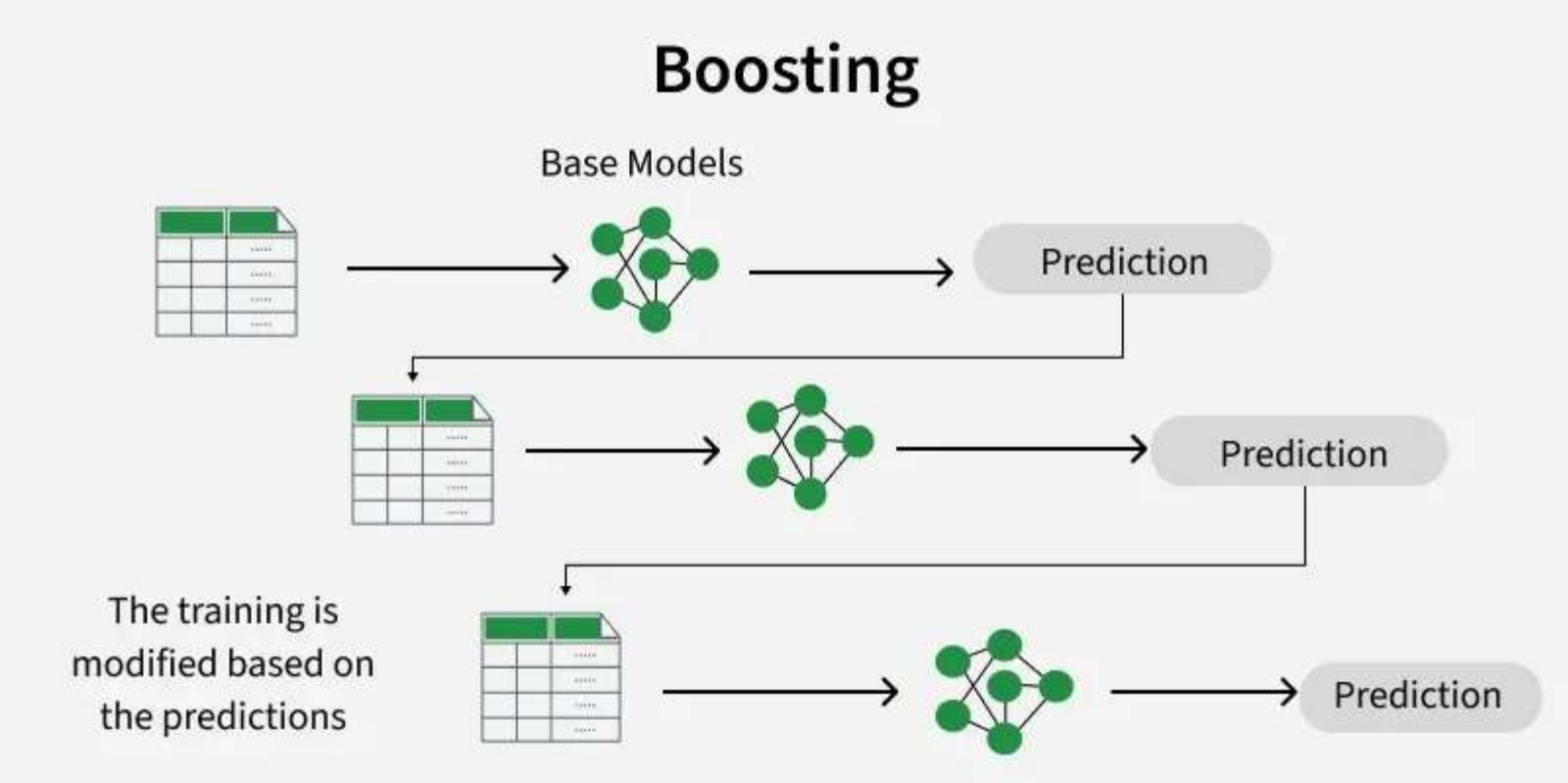
Boosting Algorithms

Boosting is a sequential ensemble machine learning technique that converts a team of weak learners (usually shallow decision trees) into a single strong learner. Unlike bagging methods like Random Forest that train trees independently in parallel, boosting algorithms train models one after the other, where **each new model is specifically designed to correct the errors made by its predecessors**


Boosting
Base Models
Prediction
Prediction
The training is modified based on the predictions
Prediction

Core Types of Boosting Algorithms

Algorithm	Core Problem It Solves	How It Works (The Mechanics)
AdaBoost	Hard-to-classify data points get ignored by standard models.	**Weight adjustment:** Increases the importance of misclassified samples so the next tree focuses on them.
GBM	Tweaking data weights doesn't mathematically minimize the global error.	**Residual learning:** Fits new trees directly to the remaining errors (gradients) of the loss function using gradient descent.
XGBoost	Gradient boosting is notoriously slow and highly prone to overfitting.	**Regularization & Speed:** Adds L1/L2 penalties to stop overfitting and uses parallel processing for rapid tree building.
LightGBM	Training on large datasets with millions of rows takes too much memory and time.	**Leaf-wise growth:** Splits trees vertically at the highest-loss leaf and bins continuous data into discrete histograms.
CatBoost	Manual encoding of categorical text variables causes data leakage and degrades accuracy.	**Ordered boosting:** Automatically processes text features using a target-encoding permutation system that prevents overfitting.

Ada boosting code

```
from sklearn.ensemble import AdaBoostRegressor
```

```
class sklearn.ensemble.AdaBoostRegressor(estimator=None, *, n_estimators=50, learning_rate=1.0, loss='linear', random_state=None)
```

XG Boosting Code

- **pip install --upgrade xgboost**
- **import xgboost as xgb**
- model = xgb.XGBRegressor(objective='reg:squarederror’,S
  n_estimators=100, random_state=42)

LG Boosting code

- **pip install lightgbm**
- **import lightgbm as lgb**
- **from lightgbm import** LGBMRegressor
- ***class*lightgbm.LGBMRegressor(*boosting_type='gbdt', num_leaves=31, max_depth= 1, learning_rate=0.1, n_estimators=100, subsample_for_bin=200000, objective=None, class_weight=None, min_split_gain=0.0, min_child_weight=0.001, min_child_samples=20, subsample=1.0, subsample_freq=0, colsample_bytree=1.0, reg_alpha=0.0, reg_lambda=0.0, random_state=None, n_jobs=- 1, silent=True, importance_type='split', **kwargs*)